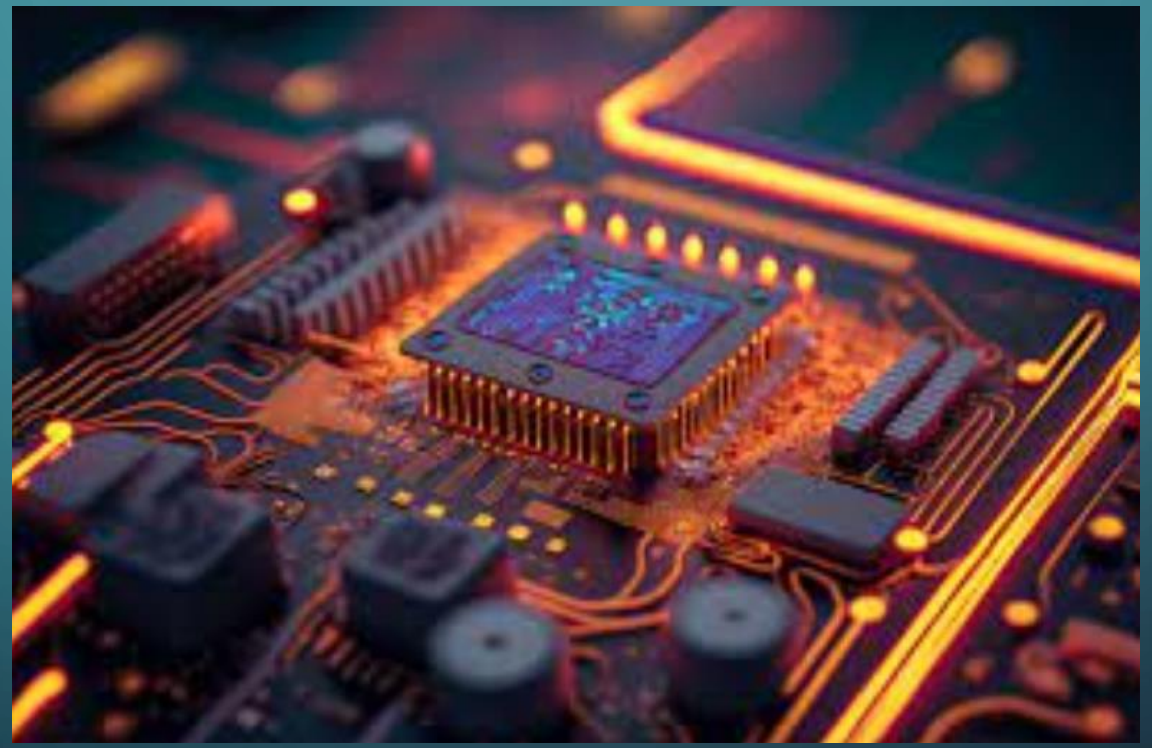
UTN
Facultad Regional Córdoba

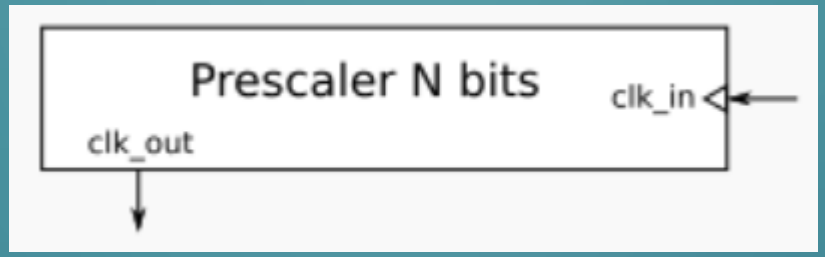
DIVISOR DE FRECUENCIA/PRESCALER EN VERILOG



Esp. Ing. Marcelo Casasnovas Año 2025

Prescaler

Los prescalers sirven para **ralentizar las señales de reloj**. Por la entrada entra una señal de reloj de frecuencia f y por la salida se obtiene una de frecuencia menor. En este tutorial haremos un **prescaler de N bits** para hacer parpadear un led a diferentes frecuencias

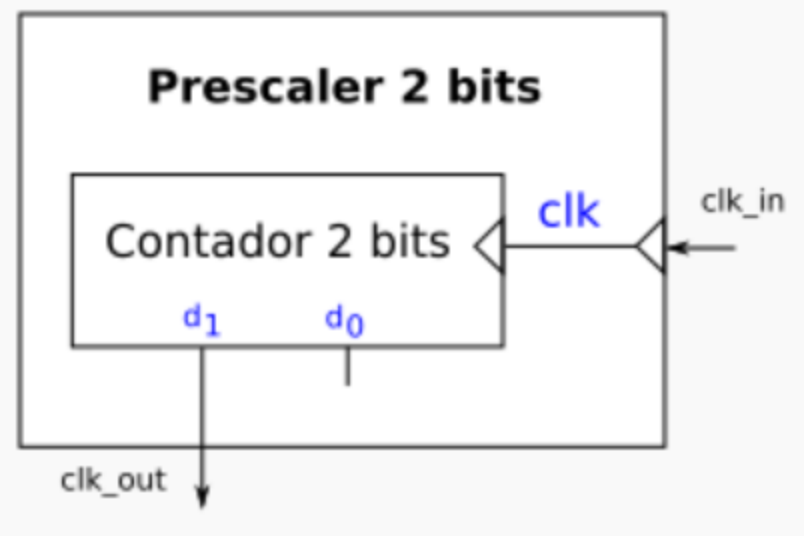


Para un prescaler de N bits, las **fórmulas** que relacionan las frecuencias y periodos de entrada con los de salida son:

	Reloj entrada	Reloj salida
Frecuencia	f_{in}	$f_{out} = \frac{f_{in}}{2^N}$
Periodo	T_{in}	$T_{out} = 2^N T_{in}$

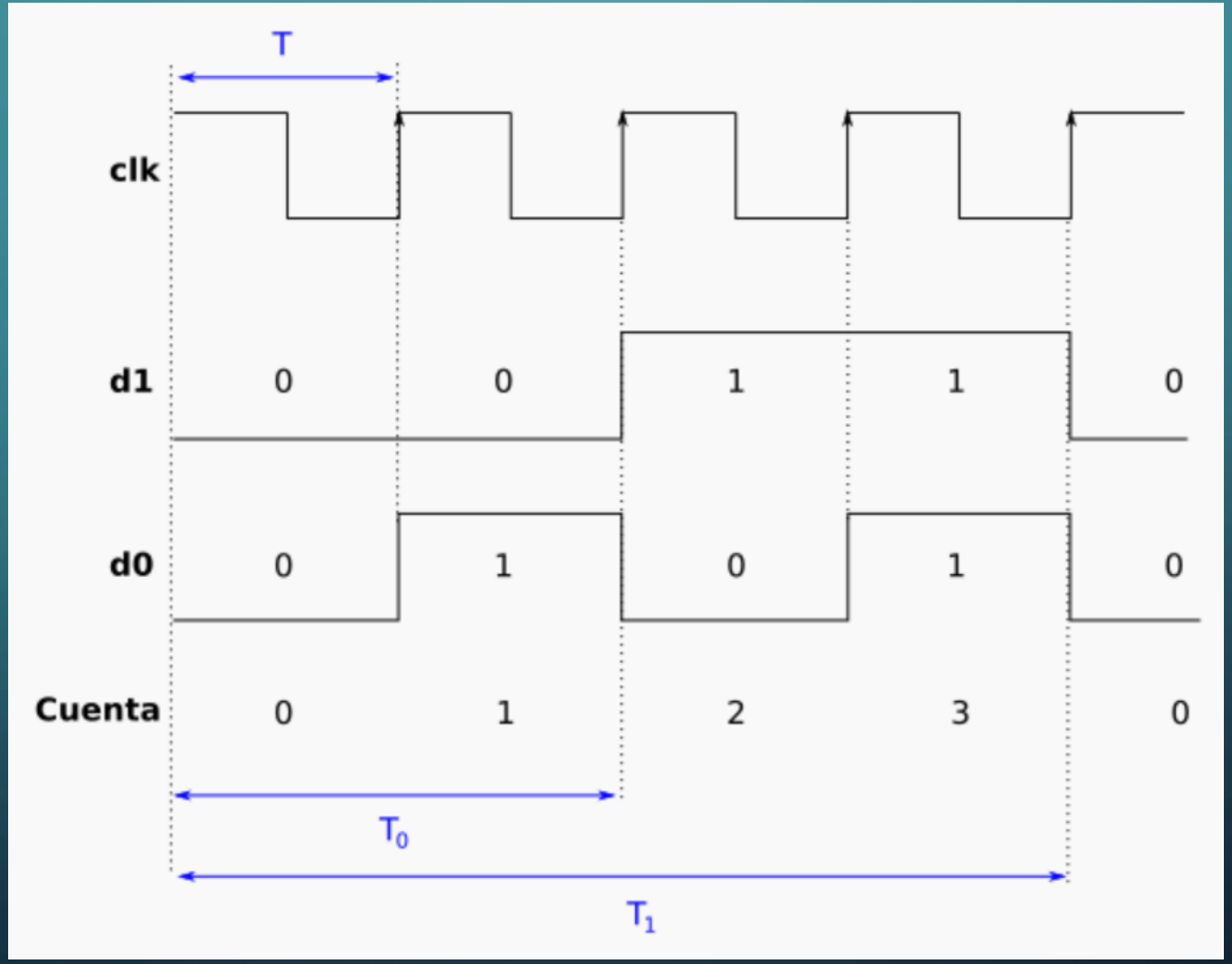
Entendiendo el prescaler de 2 bits

Antes de implementar un prescaler de N bits, vamos a entender cómo funciona uno de 2 bits



Internamente está constituido por un **contador de 2 bits**, cuyas salidas son d0 y d1. La de mayor peso es la que se saca como señal de salida. Este contador se incrementa en cada flanco de subida de clk, que tiene un periodo T. Si observamos las señales de salida de sus dos bits (d0 y d1):

Vemos que el periodo de la señal d0 es 2 veces T, y la de la señal d1 es 4 veces T. Es decir, que cada nuevo bit duplica el periodo de la señal anterior. Siguiendo la fórmula general, el periodo de este prescaler de 2 bits es: Tout = 2^2 * T = 4 * T (y gráficamente comprobamos que es así).



Prescaler de 8 bits.

Tenemos una entrada **clk** que es el reloj del sistema y un registro de salida de 8 bits que llamamos **cont** y que declaramos así: output reg [7:0] cont. Podemos hacer el símil de que un registro es un bus de datos.

Cuando declaramos una salida como un registro quiere decir que ese registro almacena bits, sin embargo, cuando declaramos con **wire** lo que hacemos es conectar entradas y salidas; wire no almacena el valor de ningún bit.

El código para el prescaler de 8 bits es el siguiente:

```
//
// Revision:
// Revision 0.01 - Ing Marcelo Casasnovas
// Additional Comments: Prescaler de 8 bits
//
//////////////////////////////////////////////////////////////////////////////////
module prescaler(
    input clk,
    output [7:0] contador
    );

//Registro contador
reg[7:0] cont;

always@(posedge clk)
begin
   cont = cont + 1;
end
assign contador = cont;
endmodule
```

Lo compilamos y subimos a la Basys 2 y lo que vemos son los 8 ledes encendidos.
¿Qué ha pasado? Lo que sucede es que como el reloj del sistema es de 50 MHz el contador se incrementa tan deprisa que los ledes aparecen siempre encendidos.

El archivo UCF con asignación de pines queda de la siguiente forma

```
1
2  # PlanAhead Generated physical constraints
3
4  NET "contador[7]" LOC = P2;
5  NET "contador[6]" LOC = P3;
6  NET "contador[5]" LOC = P4;
7  NET "contador[4]" LOC = P5;
8  NET "contador[3]" LOC = P7;
9  NET "contador[2]" LOC = P8;
10 NET "contador[1]" LOC = P14;
11 NET "contador[0]" LOC = P15;
12 NET "clk" LOC = P54;
13
```

Prescaler de 24 bits

Para ver la diferencia vamos a hacer un prescaler de 28 bits y sacaremos por los ledes, los 8 bits más significativos.

Declaramos como salidas los 8 leds de la Basys 2 dentro de module prescaler()

```
output led0, led1, led2, led3, led4, led5, led6, led7
```

Declaramos un contador de 28 bits

```
reg [28:0] cont;
```

Asignamos a cada uno de los ledes, un bit del registro cont

```
assign ledX = cont [XX];
```

Lo ponemos todo junto y el código queda así:

```
// Revision 0.01 - Ing Marcelo Casasnovas
// Additional Comments: Prescaler de 8 bits
//
//////////////////////////////////////////////////////////////////
module prescaler(
    input clk,
    output [7:0] led
    );

//Registro contador
reg[28:0] cont;

always@(posedge clk)
begin
   cont <= cont + 1;
end

assign led[0] = cont[28];
assign led[1] = cont[27];
assign led[2] = cont[26];
assign led[3] = cont[25];
assign led[4] = cont[24];
assign led[5] = cont[23];
assign led[6] = cont[22];
assign led[7] = cont[21];
endmodule
```

```
module prescaler(
    input clk,
    output [7:0] led
    );

//Registro contador
reg[28:0] cont;

always@(posedge clk)
begin
   cont <= cont + 1;
end

//assign led[0] = cont[28];
//assign led[1] = cont[27];
//assign led[2] = cont[26];
//assign led[3] = cont[25];
//assign led[4] = cont[24];
//assign led[5] = cont[23];
//assign led[6] = cont[22];
//assign led[7] = cont[21];

assign led[7:0] = cont[28:21];

endmodule
```

```
1
2  # PlanAhead Generated physical constraints
3
4  NET "led[7]" LOC = P2;
5  NET "led[6]" LOC = P3;
6  NET "led[5]" LOC = P4;
7  NET "led[4]" LOC = P5;
8  NET "led[3]" LOC = P7;
9  NET "led[2]" LOC = P8;
10 NET "led[1]" LOC = P14;
11 NET "led[0]" LOC = P15;
12 NET "clk" LOC = P54;
13
```

Lo subimos a la Basys 2 y vemos parpadear los 6 bits más significativos. ¡Hemos dividido las frecuencias hasta hacerlas visibles al ojo humano!

Contadores

Como su nombre indica los contadores sirven para contar eventos.

En los siguientes ejemplos veremos dos contadores, el primero de 4 bits que se incrementa de manera manual cada vez que pulsamos el SW1 de la Basys 2. El segundo contador -de 8 bits- se incrementa de manera automática, con una tasa que seleccionamos mediante un prescaler como los que hemos visto más arriba.

Contador de 4 bits

```
// contador manual de 4 bits por flanco de SW1
module counter(input SW1, output reg [3:0] counter);
always @(posedge SW1)
begin
  counter <= counter + 1;
end
endmodule
```

Contador automático de 8 bits.

Cómo hacer un contador automático de 8 bits. La frecuencia de refresco la seleccionaremos mediante el uso de un prescaler.

Declaramos la entrada de reloj y el contador de 8 bits como salida, para ver su valor en los 8 leds de la Basys 2

```
module counter(input clk, output reg [7:0] counter);
```

Declaramos los relojes `clk` y `clk_cont` y un registro de 24 bits para el prescaler.

```
wire clk;               // reloj del sistema
wire clk_cont;             // reloj auxiliar para el contador
reg [23:0] prescaler = 0;   // registro prescaler inicializado a 0
```

Asignamos la frecuencia de refresco al bit 17 del prescaler.

```
assign clk_cont = prescaler[17];
```

El registro prescaler se incrementa con cada flanco del reloj del sistema.

```
always @(posedge clk)
begin
  prescaler <= prescaler + 1;
end
```

Sin embargo, el contador se incrementa con cada flanco de `clk_cont`.

```
always @(posedge clk_cont)
begin
  counter <= counter + 1;
end
endmodule
```

Lo ponemos todo junto y el código queda así:

```
// Revision 0.01 - File Created
// Additional Comments:
// CONTADOR AUTOMÁTICO DE 8 BITS CON PRESCALER
//////////////////////////////////////////////////////////////////////////////////
module cont_prescl(
input clk,
output [7:0] led
);
//wire clk;
//wire clk_cont;
reg [28:0] prescaler ;
reg [7:0] counter;

//assign clk_cont = prescaler[26];
assign led[7:0] = counter [7:0];     // prescaler[28:21]

always @(posedge clk)
 begin
    prescaler <= prescaler + 1;
 end

always @(posedge prescaler[20])  // clk_cont
begin
   counter <= counter + 1;
end

endmodule
```

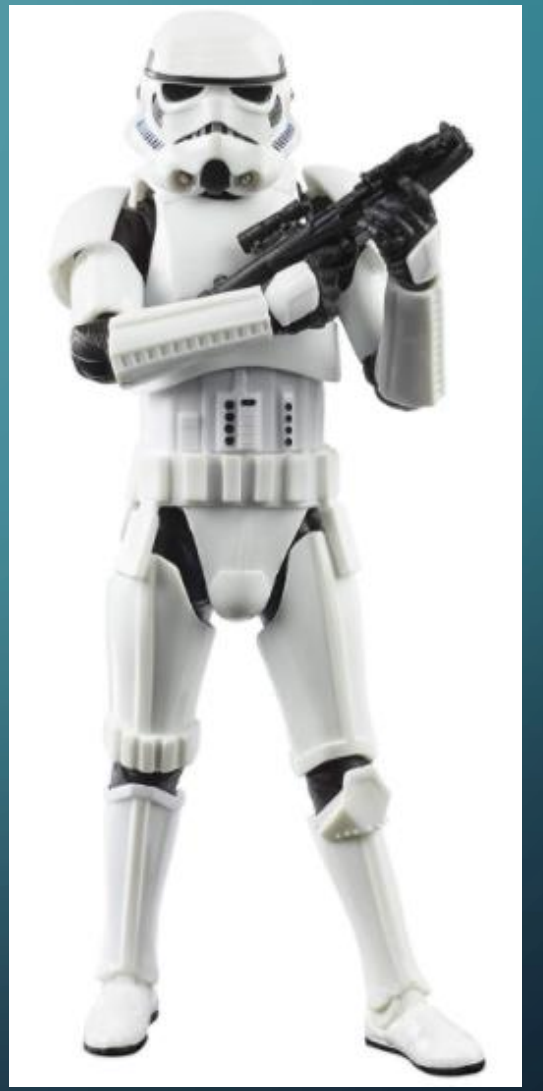
Confidential C

El archivo UCF queda:

```
NET "led[7]" LOC = P2;
NET "led[6]" LOC = P3;
NET "led[5]" LOC = P4;
NET "led[4]" LOC = P5;
NET "led[3]" LOC = P7;
NET "led[2]" LOC = P8;
NET "led[1]" LOC = P14;
NET "led[0]" LOC = P15;
NET "clk" LOC = P54;
```

Confidential C

PREGUNTAS ?????



Confidential C